

Universidade Estadual de Feira de Santana
Departamento de Tecnologia
MI - Projeto de Circuitos Digitais
Tutor: Anfranserai Moraes Dias
Circuitos Combinacionais em FPGA

Pedro Haywanon S. Araujo

Pedro Kruschewsky Lordelo

Resumo

Esse relatório descreve o desenvolvimento e a implementação de uma **ULA (Unidade Lógica Aritmética)** em **Verilog Estrutural**, utilizando o software *Quartus* em uma placa FPGA MAX DE10-Lite. No documento, são apresentados os fundamentos teóricos de álgebra booleana e arquitetura digital aplicados ao projeto, assim como a descrição e o desenvolvimento das soluções adotadas para atender aos requisitos propostos. Há também a análise e simplificação do circuito, descrição dos módulos desenvolvidos, bem como os testes e simulações realizadas. O trabalho ressalta a importância da modularização no projeto de circuitos digitais e serve como registro técnico do processo de concepção e implementação da ULA.

Palavras-chaves: relatório. desenvolvimento. análise. ULA.

1 INTRODUÇÃO

As Unidades Lógicas e Aritméticas (ULAs) representam um bloco fundamental dentro de um processador, sendo responsável pela execução das operações matemáticas e lógicas que sustentam o funcionamento de praticamente todos os sistemas computacionais modernos. Em essência, a ULA é o coração da capacidade de processamento de uma CPU, uma vez que nela ocorrem operações como soma, subtração, multiplicação, divisão e operações booleanas, que dão suporte desde cálculos simples até a execução de algoritmos complexos.

O objetivo desse projeto foi desenvolver, em linguagem Verilog estrutural, uma ULA que recebe dois valores inteiros de 4 bits cada, implementada no software Quartus e posteriormente testada na placa FPGA Intel MAX 10. O protótipo foi concebido para

realizar sete operações, sendo elas quatro aritméticas: Soma ($A + B$), Subtração ($A - B$), Multiplicação ($A \times B$) e Divisão (A / B), e três lógicas: AND ($A \& B$), OR ($A || B$) e XOR ($A \wedge B$). Além disso, contempla o uso de uma entrada de *Carry-In*, permitindo que os cálculos possam considerar operações anteriores, e saídas de quatro *flags*: (*Overflow*, Erro, Zero e *Carry-Out*), que fornecem informações adicionais sobre as condições do resultado obtido.

Outro aspecto essencial do projeto foi a forma de exibição dos resultados, limitados a valores inteiros de até dois dígitos decimais (0 a 99). Para isso, foram utilizados dois displays de 7 segmentos e os LEDs da própria placa FPGA, possibilitando uma visualização clara e intuitiva das operações realizadas. Para atingir esse objetivo, foi necessário empregar técnicas de simplificação lógica, como os Mapas de Karnaugh para a extração de expressões minimizadas no formato SOP (Soma de Produtos). Esse processo garantiu a otimização do circuito, reduzindo o número de portas lógicas e tornando o sistema o mais eficiente possível.

Dessa forma, este trabalho possibilitou aos estudantes solidificar os conhecimentos adquiridos em sala de aula. A produção desta ULA permitiu explorar os conceitos de aritmética binária, sua implementação em Verilog estrutural e a importância dessa implementação para a arquitetura digital de computadores.

2 METODOLOGIA

2.1 REFERENCIAL TEÓRICO

2.1.1 ÁLGEBRA BOOLEANA E CIRCUITOS LÓGICOS

O entendimento dos circuitos lógicos e da Álgebra Booleana foi obtido, principalmente, do livro "Elementos da Eletrônica Digital" (IDOETA, CAPUANO, 2012). Esses circuitos são desenvolvidos com base em uma notação de estado binária, distribuída entre o 0 (nível lógico baixo) e o 1 (nível lógico alto). Nesse sentido, percebe-se que os níveis lógicos têm o potencial de produzir determinadas saídas com o uso de operações, sejam elas aritméticas ou booleanas. Logo, é notável que existem apenas duas situações em cada variável da álgebra binária, e elas são opostas entre si.

2.1.2 PORTA AND

A função AND, ou função E, executa uma multiplicação booleana entre duas ou mais variáveis. A sua expressão resulta em nível lógico alto, se, e somente se, todos os bits estiverem em nível lógico alto, como é possível ver na sua tabela verdade (imagem 1). A porta AND, ou de conjunção, utilizada na ULA, é representada através do símbolo presente na imagem 2.

TABELA VERDADE - AND		
A	B	A.B
0	0	0
0	1	0
1	0	0
1	1	1

Figura 1 – Tabela verdade And. Fonte: Blogspot.com. 2025

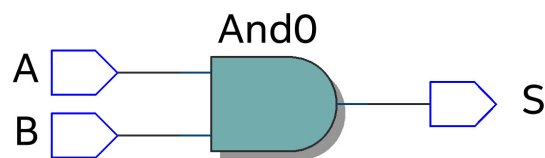


Figura 2 – Porta AND. Fonte: Autoria própria. 2025

2.1.3 PORTA OR

A função OR, ou função OU, executa uma soma booleana entre duas ou mais variáveis. A sua expressão resulta em nível lógico alto se pelo menos um bit estiver em nível lógico alto, como é possível ver na sua tabela verdade (imagem 3). A porta OR, ou de disjunção, utilizada na ULA, é representada através do símbolo presente na imagem 4.

TABELA VERDADE - OR		
A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

Figura 3 – Tabela verdade OR. Fonte: Blogspot.com. 2025

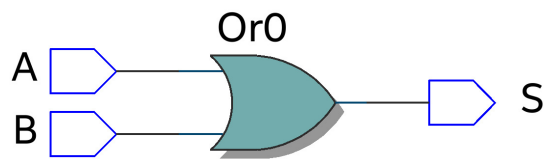


Figura 4 – Porta OR. Fonte: Autoria própria. 2025

2.1.4 PORTA NOT

A função NOT inverte o nível de tensão da saída, ou seja, se o nível lógico foi alto, a função produz saída de nível lógico baixo, e vice-versa, como é possível ver na sua tabela verdade (imagem 5). A porta NOT, ou de negação, utilizada na ULA, é representada através do símbolo presente na imagem 6.

TABELA VERDADE - NOT	
A	$\sim A$
0	1
1	0

Figura 5 – Tabela verdade NOT. Fonte: Blogspot.com. 2025.

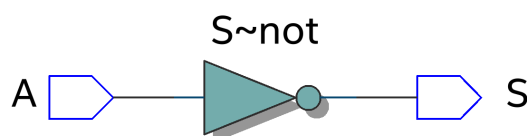


Figura 6 – Porta NOT. Fonte: Autoria própria. 2025

2.1.5 PORTA XOR

A função lógica XOR, ou OR exclusivo, é utilizada quando se deseja que a saída assumo nível lógico alto apenas nos casos em que as entradas são diferentes entre si, assim como é possível ver na sua tabela verdade (imagem 7). Portanto, a saída apresentará nível lógico alto quando o número de entradas em nível alto for ímpar. A porta XOR, utilizada na ULA, é representada através do símbolo presente na imagem 8.

TABELA VERDADE - XOR		
A	B	A#B
0	0	0
0	1	1
1	0	1
1	1	0

Figura 7 – Tabela verdade XOR. Fonte: Blogspot.com. 2025.

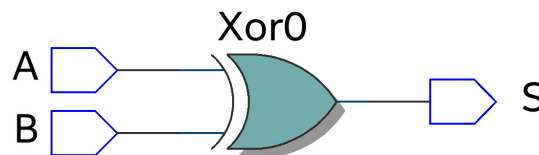


Figura 8 – Porta XOR. Fonte: Autoria própria. 2025

2.1.6 SOMADOR COMPLETO

"O Somador Completo é um circuito para efetuar a soma completa de uma coluna, considerando o transporte de entrada."(IDOETA, CAPUANO. [pág 231, 2012](#)). Em outras palavras, trata-se de um circuito digital que processa mais de uma variável, levando em conta tanto o *Carry-In*, quanto o *Carry-Out* da operação. Seu funcionamento baseia-se na propagação do *Carry-Out* de uma operação como *Carry-In* da operação subsequente. A representação do Somador Completo de 1 bit é apresentada na imagem 9.

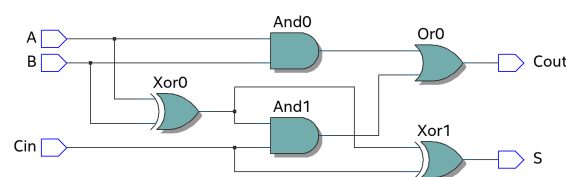


Figura 9 – Somador Completo de 1 bit. Fonte: Autoria própria. 2025

2.1.7 SUBTRATOR COMPLETO

"O Subtrator Completo é um circuito que efetua a subtração completa de uma coluna, ou seja, considera o transporte de entrada proveniente da coluna anterior."(IDOETA, CAPUANO, [pág. 237, 2012](#)). Em outras palavras, trata-se de um circuito digital que processa mais de uma variável, levando em conta tanto o *Borrow-In*, quanto o *Borrow-Out* da operação. Seu funcionamento baseia-se na propagação do *Borrow-Out* de uma operação como *Borrow-In* da operação subsequente. A representação RTL do Subtrator Completo de 1 bit é apresentada na imagem 10.

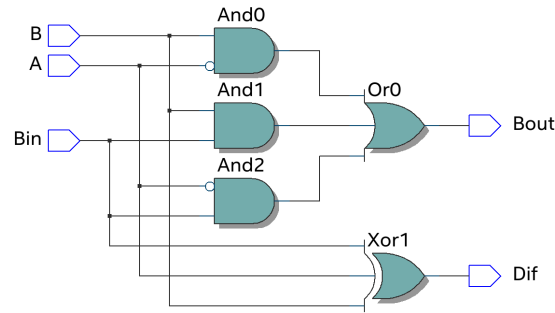


Figura 10 – Subtrator Completo de 1 bit. Fonte: Autoria própria. 2025

2.1.8 MULTIPLICADOR

A multiplicação aritmética, também denominada *Matriz de Duplo Encadeamento* (IDOETA, CAPUANO, 2012), consiste em um processo de geração de produtos parciais (também chamados produtos canônicos), obtidos por meio de portas *AND* e somadores completos. O princípio de funcionamento é análogo à operação de multiplicação em base decimal no formato de “conta armada”, em que os produtos parciais são gerados linha a linha e, posteriormente, somados. Dessa forma, a combinação desses produtos parciais resulta no produto final. A representação RTL do Multiplicador é apresentada na imagem 11.

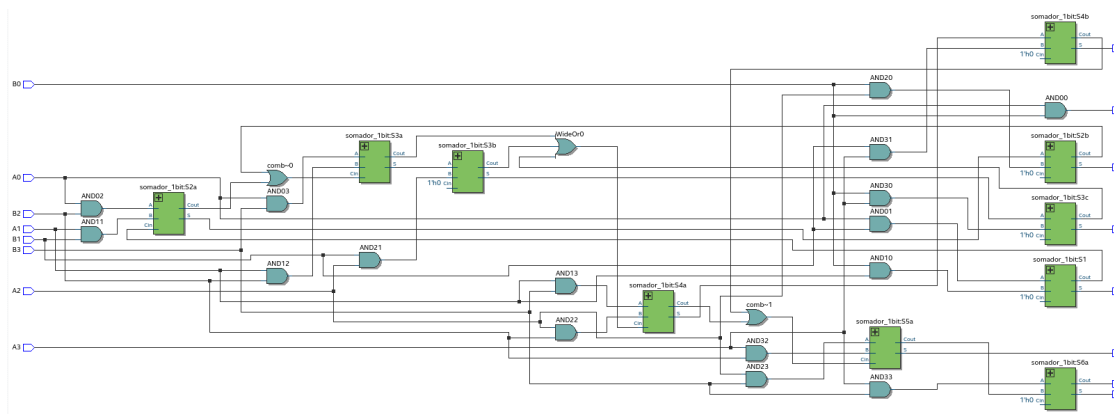


Figura 11 – Multiplicador. Fonte: Autoria própria. 2025

2.1.9 DIVISOR

O processo de divisão binária é análogo ao de números decimais, no qual é necessário verificar se o divisor “cabe” no dividendo antes de realizar a subtração. Dessa forma, a implementação de uma divisão booleana requer o uso de subtratores completos entre as parcelas da divisão e multiplexadores para selecionar a saída apropriada TOCCI, 2011. A representação RTL do Divisor é apresentada na imagem 12.

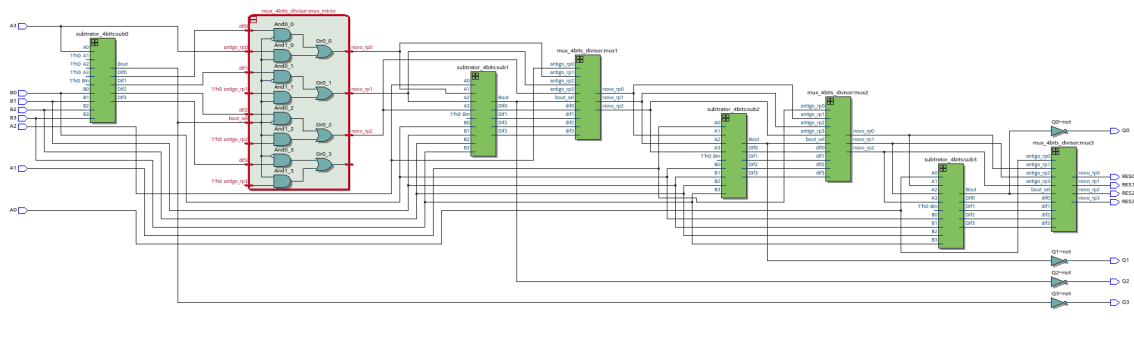


Figura 12 – Divisor. Fonte: Autoria própria. 2025.

2.1.10 MULTIPLEXADOR

"Os circuitos multiplex são utilizados nos casos em que precisamos enviar um certo número de informações, contidas em vários canais, a um só canal." (IDOETA, CAPUANO, [pág. 372, 2012](#)). Ou seja, os multiplexadores, ou apenas MUX, são portas de comutação, responsáveis por escolher uma entre várias entradas e encaminhá-la para uma única saída de acordo com os sinais de seleção previamente definidos. Para exemplificar, o multiplexador 2:1 é o modelo mais simples, possuindo duas entradas de dados, uma linha de seleção e uma saída. A escolha da entrada a ser encaminhada para a saída depende exclusivamente do valor lógico aplicado ao sinal de seleção, como pode ser observada em sua tabela verdade na imagem [13](#).

S	A	B	Out
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Figura 13 – Tabela verdade multiplexador. Fonte: Adaptado de ResearchGate (2018).

3 DESENVOLVIMENTO

O protótipo foi implementado na placa FPGA DE10-Lite, utilizando Verilog estrutural no software Quartus. A ULA foi organizada em módulos lógicos para cada operação, com um multiplexador controlando a seleção do resultado. Flags de condição foram incluídas para situações específicas das operações, e os resultados são exibidos nos LEDs e em dois displays de 7 segmentos. Todas as expressões lógicas foram previamente simplificadas, garantindo a otimização e eficiência do circuito.

3.1 SOMADOR 4 BITS

O Somador que opera com duas entradas de 4 bits foi feito a partir da tabela verdade do somador de 1 bit (tabela 1). Essa tabela, por sua vez, foi construída considerando três possíveis entradas: o bit A , o bit B e o bit de transporte de entrada (C_{in}). O funcionamento segue o mesmo princípio da soma em base decimal: cada coluna soma seus dígitos e pode ou não gerar um transporte para a próxima coluna mais significativa.

No caso binário, a soma dos três bits de entrada gera dois resultados possíveis:

- O **bit de soma** (S), que corresponde ao bit menos significativo do resultado dessa soma;
- O **bit de transporte de saída** (C_{out}), que corresponde ao bit mais significativo do resultado dessa soma.

O Somador de 4 bits foi feito a partir da instanciación do circuito extraído da expressão SOP (Soma Dos Produtos) em cascata, de modo que o C_{out} da operação atual se torne a entrada C_{in} da próxima operação.

A	B	C_{in}	RESULTADO	BINÁRIO	C_{out}	S
0	0	0	0	00	0	0
0	0	1	1	01	0	1
0	1	0	1	01	0	1
0	1	1	2	10	1	0
1	0	0	1	01	0	1
1	0	1	2	10	1	0
1	1	0	2	10	1	0
1	1	1	3	11	1	1

Tabela 1 – Tabela verdade do somador de 1 bits. Fonte: Autoria própria. 2025.

Após isso, foi necessário retirar duas expressões: quando o C_{out} possui nível lógico alto e quando a Saída (S) possui nível lógico alto.

3.1.0.1 Expressão de C_{out}

O C_{out} , ou *Carry-Out*, é o bit de transporte gerado quando a soma de dois bits e o *Carry-In* (C_{in}) excede a capacidade de um único bit. Ele indica que houve *overflow* na coluna corrente da soma e deve ser propagado para a próxima coluna mais significativa. Equivalentemente, C_{out} assume nível lógico alto sempre que *pelo menos dois* dos três bits de entrada (A, B, C_{in}) forem iguais a 1.

A partir da Tabela 1, observa-se que $C_{out} = 1$ nas combinações:

$$(0, 1, 1), \quad (1, 0, 1), \quad (1, 1, 0), \quad (1, 1, 1).$$

Escrevendo a soma dos mintermos correspondentes, obtém-se a forma canônica:

$$C_{out} = A'BC_{in} + AB'C_{in} + ABC'_{in} + ABC_{in}.$$

Simplificação passo a passo:

$$\begin{aligned}C_{out} &= A'BC_{in} + AB'C_{in} + ABC'_{in} + ABC_{in} \\&= A'BC_{in} + AB'C_{in} + AB(C'_{in} + C_{in}) \\&= A'BC_{in} + AB'C_{in} + AB \\&= AB + C_{in}(A'B + AB') \\&= AB + C_{in}(A \oplus B).\end{aligned}$$

Assim, uma forma compacta para implementação é:

$$C_{out} = AB + C_{in} (A \oplus B).$$

Outra forma equivalente quando se deseja apenas portas AND/OR, é:

$$C_{out} = AB + AC_{in} + BC_{in}.$$

Mapa de Karnaugh:

$A \backslash BC$	00	01	11	10
0	0	0	1	0
1	0	1	1	1

Agrupamentos possíveis:

- Coluna $BC = 11$ (linhas $A = 0$ e $A = 1$) $\rightarrow BC_{in}$;
- Linha $A = 1$, colunas 10 e 11 $\rightarrow AB$;
- Linha $A = 1$, colunas 01 e 11 $\rightarrow AC_{in}$.

Combinando os grupos, obtemos novamente:

$$C_{out} = AB + AC_{in} + BC_{in}.$$

3.1.0.2 Expressão de S

O bit de soma S de um somador completo representa o bit menos significativo do resultado da soma entre A , B e C_{in} . Observando a Tabela 1, $S = 1$ nas seguintes combinações de entradas:

$$(0, 0, 1), \quad (0, 1, 0), \quad (1, 0, 0), \quad (1, 1, 1).$$

Escrevendo a soma dos mintermos correspondentes, obtém-se a forma canônica:

$$S = A'B'C_{in} + A'BC'_{in} + AB'C'_{in} + ABC_{in}.$$

Simplificação passo a passo:

$$\begin{aligned}
 S &= A'B'C_{in} + A'BC'_{in} + AB'C'_{in} + ABC_{in} \\
 &= A'(B'C_{in} + BC'_{in}) + A(B'C'_{in} + BC_{in}) \\
 &= A'(B \oplus C_{in}) + A(B \oplus C_{in})' \\
 &= (A \oplus B \oplus C_{in}).
 \end{aligned}$$

Ou seja, a forma mais enxuta da expressão que descreve S é::

$$S = A \oplus B \oplus C_{in}$$

Mapa de Karnaugh

$A \backslash BC$	00	01	11	10
0	0	1	0	1
1	1	0	1	0

Agrupamentos possíveis:

- Não existem agrupamentos possíveis nesse mapa.
- Cada 1 aparece isolado, o que é típico de funções de paridade.

Portanto, não há simplificação adicional pelo mapa de Karnaugh. A função permanece como soma de mintermos:

$$S = A'B'C_{in} + A'BC'_{in} + AB'C'_{in} + ABC_{in}.$$

Porém, reconhecendo o padrão de paridade, obtemos a forma mais compacta:

$$S = A \oplus B \oplus C_{in}.$$

3.2 SUBTRATOR 4 BITS

O subtrator que opera sobre duas entradas de 4 bits é feito utilizando um bit de empréstimo de entrada (*Borrow-In*) em cada estágio, gerando como saída os 4 bits de diferença e o bit de empréstimo de saída (*Borrow-Out*).

O projeto foi baseado na instanciação de 4 subtratores de 1 bit em cascata, de modo que o *Bout* da operação anterior se torne o *Bin* da operação seguinte. Dessa forma, a subtração ocorre exatamente como na subtração manual na base decimal.

A	B	<i>Bin</i>	RESULTADO	BINÁRIO	<i>Bout</i>	Dif
0	0	0	0	00	0	0
0	0	1	1	01	1	1
0	1	0	1	01	1	1
0	1	1	0	00	1	0
1	0	0	1	01	0	1
1	0	1	0	00	0	0
1	1	0	0	00	0	0
1	1	1	1	01	1	1

Tabela 2 – Tabela verdade do subtrator de 1 bit. Fonte: Autoria própria. 2025.

3.2.1 Expressão *Bin*

O bit de empréstimo de saída (*Bout*) é gerado quando a subtração de *B* e *Bin* de *A* exige pegar emprestado de uma coluna mais significativa. Observando a Tabela 2, *Bout* = 1 nas combinações:

$$(0, 0, 1), \quad (0, 1, 0), \quad (0, 1, 1), \quad (1, 1, 1).$$

Escrevendo a soma dos mintermos, obtemos a forma canônica:

$$Bout = A'B Bin' + A'B' Bin + A'B Bin + AB Bin$$

Simplificação:

$$\begin{aligned}
Bout &= A'B(Bin' + Bin) + A'B' Bin + AB Bin \\
&= A'B + A'B' Bin + AB Bin \\
&= A'B + Bin(A'B' + AB) \\
&= A'B + Bin(A \oplus B)'
\end{aligned}$$

Mapa de Karnaugh de Bout

$A \backslash BC$	00	01	11	10
0	0	1	1	1
1	0	1	1	0

Agrupamentos possíveis:

- Coluna BC=11 $\rightarrow B \cdot Bin$
- Linha A=0, colunas 01 e 10 $\rightarrow A'B + A'Bin$

Forma simplificada confirmada: $Bout = A'B + A'Bin + B \cdot Bin$.

3.2.2 Expressão *Dif*

O bit de diferença (*Dif*) é o resultado da subtração do bit A menos B e Borrow-In. Observando a tabela, *Dif* = 1 nas combinações:

$$(0, 0, 1), \quad (0, 1, 0), \quad (1, 0, 0), \quad (1, 1, 1)$$

A forma canônica é:

$$Dif = A'B'Bin + A'B Bin' + AB' Bin' + AB Bin$$

Simplificação:

$$\begin{aligned} Dif &= A'B'Bin + A'B Bin' + AB' Bin' + AB Bin \\ &= A'(B \oplus Bin) + A(B \oplus Bin)' \\ &= A \oplus B \oplus Bin \end{aligned}$$

Portanto, a forma mais enxuta da expressão que descreve o bit de diferença é:

$$\boxed{Dif = A \oplus B \oplus Bin}$$

Mapa de Karnaugh

$A \backslash BC$	00	01	11	10
0	0	1	0	1
1	1	0	1	0

Observações:

- Cada 1 aparece isolado, típico de funções de paridade.
- Não há agrupamentos possíveis para simplificação adicional.

3.3 OPERAÇÃO LÓGICA AND

A operação lógica *AND* em operandos de 4 bits foi realizada bit a bit, assim como o Somador e o Subtrator. Essa implementação foi feita por meio da instancição do módulo *AND* de 1 bit (Figura 2) quatro vezes, formando o módulo *AND* de 4 bits. Cada instância processa um par de bits equivalentes das entradas, conforme demonstrado na tabela verdade apresentada na Figura 1.

3.4 OPERAÇÃO LÓGICA OR

A operação lógica *OR* em operandos de 4 bits foi realizada bit a bit, assim como o Somador e o Subtrator. Essa implementação foi feita por meio da instancição do módulo *OR* de 1 bit (Figura 4) quatro vezes, formando o módulo *OR* de 4 bits. Cada instância processa um par de bits equivalentes das entradas, conforme demonstrado na tabela verdade apresentada na Figura 3.

3.5 OPERAÇÃO LÓGICA XOR

A operação lógica *XOR* em operandos de 4 bits foi realizada bit a bit, assim como o Somador e o Subtrator. Essa implementação foi feita por meio da instanciação do módulo *XOR* de 1 bit (Figura 8) quatro vezes, formando o módulo *XOR* de 4 bits. Cada instância processa um par de bits equivalentes das entradas, conforme demonstrado na tabela verdade apresentada na Figura 7.

3.6 MULTIPLICAÇÃO

A multiplicação de duas palavras de 4 bits foi realizada a partir da combinação de **somadores completos** e **portas AND**, seguindo o método conhecido como *Matriz de Duplo Encadeamento*. Nesse método, a soma dos produtos parciais resulta no produto final de 8 bits. O funcionamento é análogo ao da multiplicação decimal, porém, no caso binário, o *Carry-In* de uma coluna deve ser ligado ao produto parcial da coluna subsequente. Cada estágio da operação envolve:

- Geração dos produtos parciais por meio de portas *AND* bit a bit;
- Soma dos produtos parciais utilizando somadores completos;
- Propagação do *carry* entre os estágios de soma.

O produto final é representado por $P_7P_6P_5P_4P_3P_2P_1P_0$, sendo P_0 o bit menos significativo e P_7 o bit mais significativo.

A Figura 14 apresenta um esquema ilustrativo da operação de multiplicação binária, enquanto o circuito RTL do multiplicador completo está apresentado na Figura 11, o que facilita a compreensão da lógica utilizada.

$$\begin{array}{rcl} & 101_2 & \rightarrow 5_{10} \\ \times & 110_2 & \rightarrow 6_{10} \\ \hline & 000 & \\ & 101 & \\ & 101 & \\ \hline & 11110_2 & \rightarrow 30_{10} \end{array}$$

Figura 14 – Representação da multiplicação binária. Fonte: Blogspot.com. 2025

3.6.1 Tabela verdade

A tabela verdade completa do multiplicador de 4x4 bits pode ser consultada no seguinte link, visto que não coube integralmente nesta documentação:

[Acessar tabela verdade do multiplicador 4x4 bits](#)

3.7 DIVISÃO

A divisão de duas palavras de 4 bits foi implementada a partir da combinação de **subtratores completos** e **multiplexadores** de 4 bits com seletor de 1 bit. O princípio de funcionamento segue o mesmo da divisão decimal: verificar se o divisor “cabe” no dividendo parcial para realizar ou não a subtração.

O processo inicia-se pela análise do bit mais significativo do dividendo (A_3). Para isso, adicionam-se três bits de valor zero (aterrados) à esquerda, formando a primeira operação de subtração ($000A_3 - B_3B_2B_1B_0$). Se o *Borrow-Out* resultante for igual a 1 (nível lógico alto), significa que o divisor não pôde ser subtraído do dividendo parcial, e, portanto, o bit do quociente é 0. Caso contrário, a subtração é válida, e o bit do quociente é 1.

Nesse sentido, é perceptível que o quociente é formado pela negação do sinal de *Borrow-Out*, de modo que:

- $Borrow-Out = 0 \Rightarrow$ subtração possível $\Rightarrow$ quociente = 1
- $Borrow-Out = 1 \Rightarrow$ subtração não possível $\Rightarrow$ quociente = 0

Seguidamente, um multiplexador seleciona, a partir do *Borrow-Out*, entre o resultado da subtração e o valor anterior do dividendo parcial. A saída desse multiplexador corresponde ao novo resto parcial da divisão, ao qual é acrescentado o próximo bit do dividendo (A_2 , A_1 , e por fim A_0), repetindo-se o procedimento até que todos os bits tenham sido processados.

As saídas do divisor correspondem ao quociente ($Q_3Q_2Q_1Q_0$), em ordem de significância, e ao resto final ($Res_3Res_2Res_1Res_0$). Contudo, os restos não foram implementados na ULA da equipe, visto que não havia obrigatoriedade de sua utilização na resolução do problema proposto. Dessa forma, o resultado desse módulo exibido como saída da ULA corresponde a uma divisão inteira.

O circuito RTL (Figura 12) da divisão pode ajudar na visualização e entendimento da lógica da divisão binária. Além disso, será apresentado na Figura 15 o circuito RTL do multiplexador do divisor.

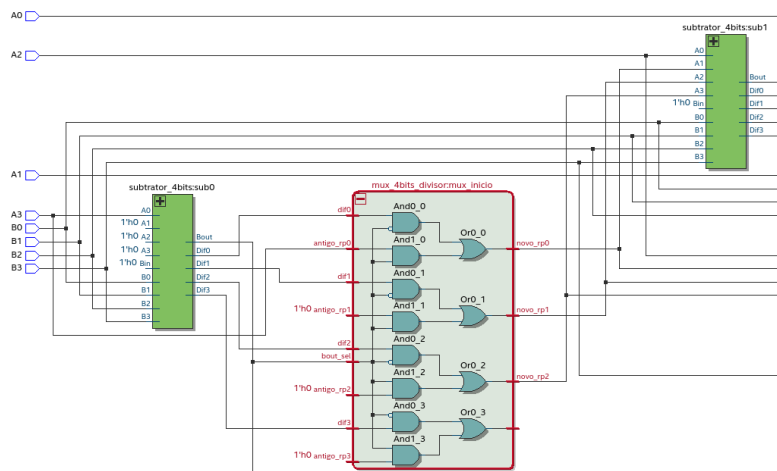


Figura 15 – Recorte do divisor com foco no multiplexador. Fonte: Autoria própria. 2025

3.7.1 Tabela verdade

Visto que o circuito não foi extraído de uma tabela verdade completa com mintermos e expressão SOP, será exibida apenas uma parte representativa da tabela verdade da divisão, com fins ilustrativos e de apoio.

A3A2A1A0	B3B2B1B0	Q3Q2Q1Q0	Res3Res2Res1Res0
1000	0010	0100	0000
1011	0011	0011	0000
0110	0011	0010	0000
1111	0100	0011	0011
1101	0010	0110	0001
0101	0010	0010	0001
1001	0011	0011	0000
1110	0101	0010	0010

Tabela 3 – Parte da tabela verdade do divisor de 4 bits. Fonte: Autoria própria. 2025.

3.8 MULTIPLEXADOR

O multiplexador de 1 bit da ULA principal foi projetado para selecionar, dentre as diferentes operações realizadas, qual resultado será encaminhado como saída final da ULA. Esse multiplexador possui sete entradas correspondentes às operações: **SOMA (S)**, **SUBTRAÇÃO (Dif)**, **MULTIPLICAÇÃO (P)**, **DIVISÃO (Q)**, **AND**, **OR** e **XOR**.

A seleção da operação é controlada por três bits de seleção: E_0 , E_1 e E_2 , sendo E_0 o bit menos significativo e E_2 o mais significativo. A organização das combinações dos bits de seleção é definida conforme a Tabela 4:

Seletores (E_2 E_1 E_0)	Operação Selecionada
000	SOMA
001	SUBTRAÇÃO
010	MULTIPLICAÇÃO
011	DIVISÃO
100	AND
101	OR
111	XOR

Tabela 4 – Configuração das chaves seletoras do multiplexador. Fonte: Autoria Própria. 2025.

Para atender a todas as operações da ULA, o multiplexador de 1 bit foi replicado oito vezes, formando o módulo *multiplexador_8bits*. Esse módulo recebe como entradas:

- 5 fios da operação de soma (4 bits da soma + Carry-Out);
- 5 fios da operação de subtração (4 bits da diferença + Borrow-Out);
- 8 fios da operação de multiplicação (produto de 8 bits);

- 4 fios da operação de divisão (quociente);
- 4 fios de cada operação lógica (AND, OR, XOR).

Ao todo, o módulo recebe 34 fios de entrada mais os 3 fios de seleção, produzindo uma saída de 8 bits. Essa decisão foi tomada considerando que a multiplicação binária é a operação com maior número de bits de saída na ULA, sendo assim garantida a apresentação completa do resultado de qualquer operação sem perda de informação.

Parte do circuito RTL, mais especificamente as saídas R0 e R1, do multiplexador principal pode ser vista na Figura 16, ilustrando a organização e a interligação dos módulos.

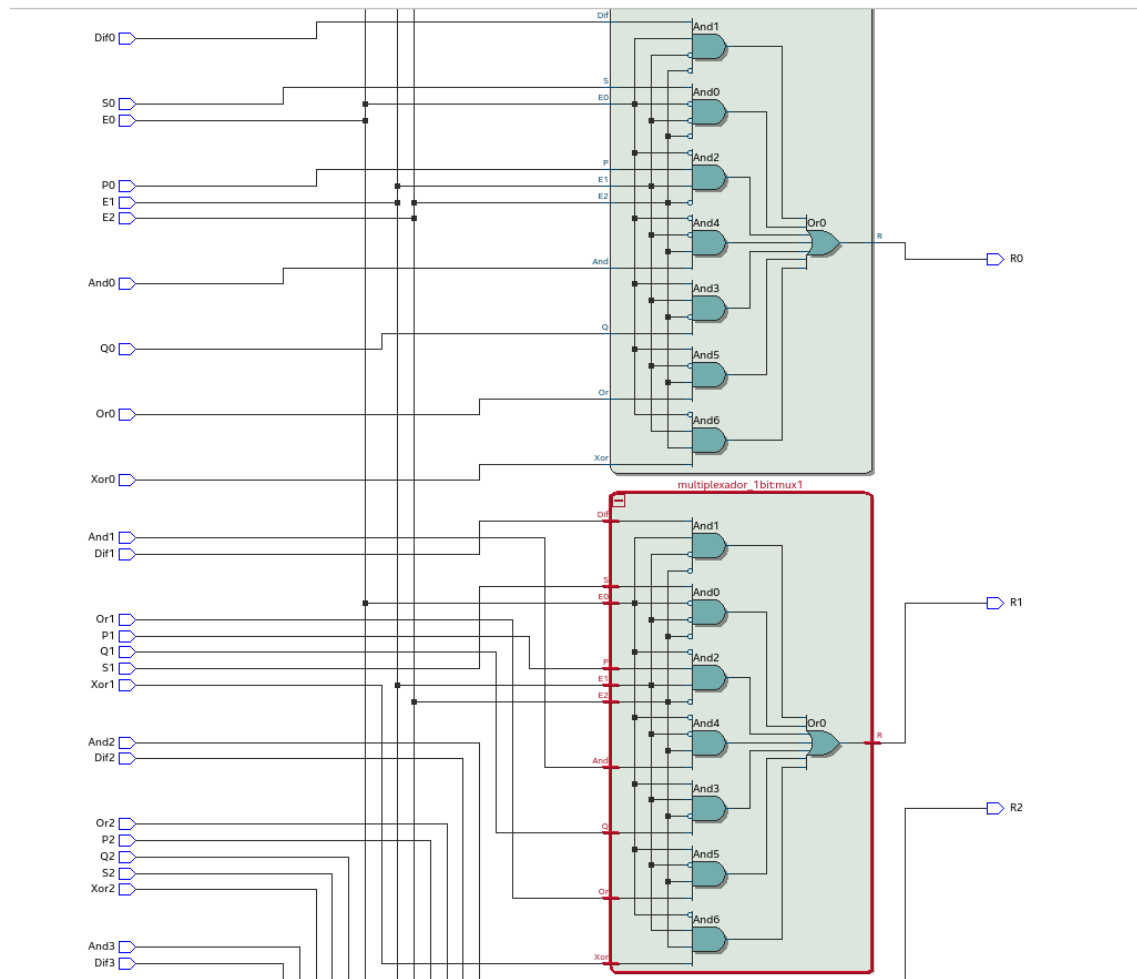


Figura 16 – Circuito RTL multiplexador recortado. Fonte: Autoria Própria. 2025

As saídas do multiplexador de 8 bits na ULA serão os resultados em binário da operação selecionada, sendo, então pinadas nos LEDs da placa.

3.9 FLAGS

As flags são indicadores que sinalizam certas condições do resultado de uma operação. No projeto, elas acendem LEDs específicos quando a condição correspondente é atendida:

- A flag Carry Out (*Cout*) acende o LED LEDR4 .
- A flag Erro acende o LED PIN_A11
- A flag Zero acende o LED PIN_B11
- A flag Overflow (OF) acende simultaneamente os LEDs do Erro e do Zero (PIN_A11 e PIN_B11)

Os tópicos abaixo estão destinados a explicar detalhadamente as condições que acionam as flags.

3.9.1 Carry-Out (COUT)

O sinal *Carry-Out* indica quando há um estouro da capacidade de um somador, sinalizando que o resultado excedeu o número máximo permitido de **4 bits**. No caso do subtrator, o sinal equivalente é chamado *Borrow-Out*, indicando que ocorreu um empréstimo durante a operação [MANO, 2012](#).

Sob essa ótica, a flag de *Carry-Out* deve ser ativada quando o somador produz um resultado maior do que a capacidade disponível (exigindo 5 bits para representação), ou quando o subtrator termina a operação com um empréstimo (*Borrow-Out*), também ultrapassando o limite de 4 bits. Essa flag é utilizada como entrada do multiplexador, estando localizada no bit R_4 , ou seja, sendo o quinto bit de saída tanto do somador quanto do subtrator.

3.9.2 Zero

A flag **zero** indica que o resultado da operação é igual a zero. Essa flag foi implementada por meio de uma operação NOR de todos os bits de saída, ou seja, é acionada quando todas as saídas da ULA são zero.

3.9.3 Erro

A flag erro foi desenvolvida para sinalizar quando há divisão por zero. Para isso, foi criado um módulo que recebe os bits do operando B (B3, B2, B1, B0) e, com a utilização de portas AND, verifica se todos os bits estão em nível lógico baixo. Se divisor for igual a zero, a flag é acionada, acendendo o LED correspondente e fornecendo uma indicação visual imediata de que ocorreu um erro na operação. Essa implementação permite que o sistema detecte e sinalize de forma eficiente condições de erro, garantindo maior segurança e confiabilidade durante a execução das operações da ULA.

3.9.4 Overflow (OF)

A flag overflow (OF) foi implementada para indicar quando o resultado de uma operação aritmética excede a capacidade de 4 bits. Nesse caso, o LED correspondente ao overflow é acionado, alertando para a situação de excesso de valor.

É importante notar que, mesmo quando a flag de overflow está acesa, o display da placa continua representando corretamente números de até dois dígitos, permitindo a visualização de resultados de 0 a 99, que é o máximo representável em dois dígitos decimais. Dessa forma, a flag serve como um indicador de limite de 4 bits, sem impedir a exibição de valores maiores no display.

3.10 DISPLAY DE 7 SEGMENTOS

O display de 7 segmentos é um dispositivo usado, normalmente, para representar números decimais. Ele é composto por 7 segmentos de LED identificados por $a0, b0, c0, d0, e0, f0, g0$ (unidade) e $a1, b1, c1, d1, e1, f1, g1$ (dezena), além do ponto (*dot*), como representado na Figura 17. Esses segmentos podem ser ativados ou não para representar os dígitos de 0 a 9, sendo que a partir do 99 o display aparece um aviso de overflow (OF). O acionamento segue o padrão comum de displays: o segmento acende quando recebe nível lógico baixo (0) e permanece apagado quando recebe nível lógico alto (1).

Display de 7 segmentos

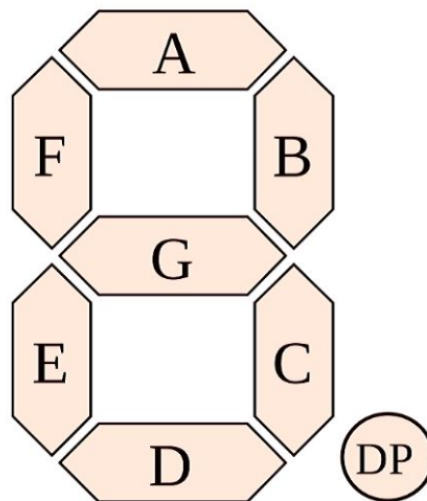


Figura 17 – Display 7 segmentos. Fonte: actualidadtecnologica.com 2025.

O projeto do display foi desenvolvido a partir da tabela verdade de cada segmento, considerando os dígitos em que cada um deveria acender ou permanecer apagado. Tomando como exemplo o segmento $g0$, ele permanece apagado para os números 0, 1 e 7, e acende para os demais. A partir dessa condição, elaborou-se a tabela verdade correspondente, na qual o valor lógico baixo (0) indica acendimento.

Binário	Decimal	g0
0000	0	1
0001	1	1
0010	2	0
0011	3	0
0100	4	0
0101	5	0
0110	6	0
0111	7	1
1000	8	0
1001	9	0

Tabela 5 – Tabela verdade do segmento $g0$ do display de 7 segmentos.

Analisando a Tabela 5, observa-se que o segmento $g0$ permanece apagado apenas quando o dígito exibido é 0, 1 ou 7. A partir dessa informação, foi utilizado o site [Kmap Solver \(Charlie Coleman\)](#) para construir o Mapa de Karnaugh (Figura 18) e extrair a expressão simplificada no formato SOP para esse segmento.

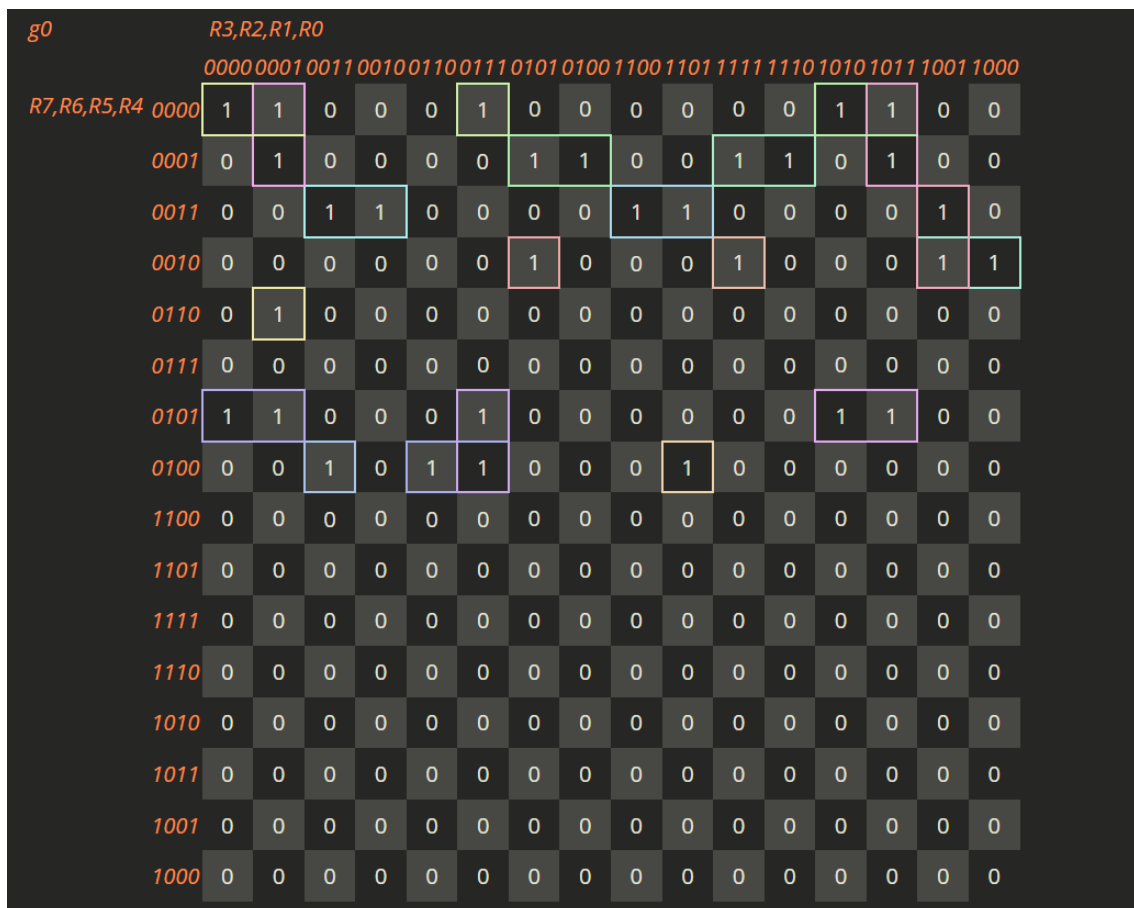


Figura 18 – Mapa de Karnaugh do segmento $g0$. Fonte: charlie-coleman.com 2025.

Dessa forma, o mesmo procedimento foi aplicado a todos os segmentos das posições de dezena e unidade, obtendo-se as respectivas expressões SOP. Ademais, o projeto foi desenvolvido para ser exposto apenas em 2 displays, ou seja, o número máximo

4 RESULTADOS E DISCUSSÕES

Para avaliar a eficiência do módulos desenvolvidos, foram realizados testes com diferentes combinações de entradas. Esses testes permitem verificar o correto funcionamento de determinado módulo em condições variadas e garantir que as flags associadas sejam acionadas corretamente. Dessa forma, é possível analisar o resultado dos testes usando a função *Simulation* do *Quartus*, que gera uma simulação das saídas com determinadas entradas.

4.1 TESTES

4.1.1 MÓDULO SOMADOR DE 4 BITS NA ULA

Os testes realizados para avaliar a eficiência do somador foram:

- $0001 + 0001 \dots$ Resultado: 0010 ($1 + 1 = 2$)
- $0000 + 0000 \dots$ Resultado: 0000 ($0 + 0 = 0$, flag ZERO)
- $1111 + 1111 \dots$ Resultado: 11110 ($15 + 15 = 30$, flags OVERFLOW e CARRY-OUT)
- $0111 + 0011 + \text{Carry-in} \dots$ Resultado: 1011 ($7 + 3 + 1 = 11$)

Utilizando a ferramenta *Simulation* do *Quartus*, é possível confirmar os resultados dos testes (19)

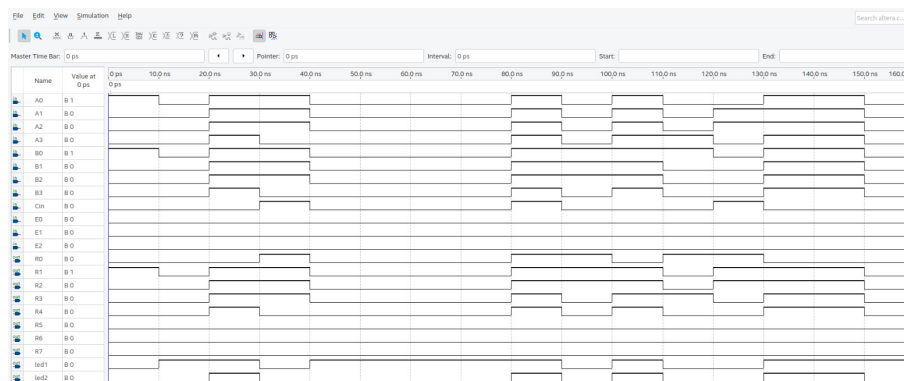


Figura 19 – Teste somador de 4 bits. Fonte: Autoria própria. 2025.

4.1.2 MÓDULO SUBTRATOR DE 4 BITS NA ULA

Os testes realizados para avaliar a eficiência do subtrator foram:

- $0001 - 0001 \dots$ Resultado: 0000 ($1 - 1 = 0$, flag ZERO)

- 0000 - 0001 ... Resultado: 1111 ($0 - 1 = 31$, flags OVERFLOW e BORROW-OUT)
- 1111 - 0010 - Borrow-in ... Resultado: 1100 ($15 - 2 - 1 = 12$)
- 1110 - 0110 - Borrow-in ... Resultado: 0111 ($14 - 6 - 1 = 7$)

Utilizando a ferramenta *Simulation* do *Quartus*, é possível confirmar os resultados dos testes (20)

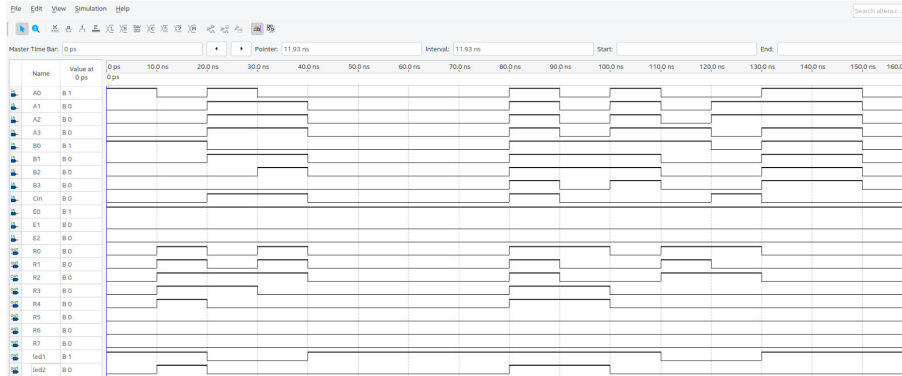


Figura 20 – Teste subtrator de 4 bits. Fonte: Autoria própria. 2025.

4.1.3 MÓDULO MULTIPLICADOR DE 4 BITS NA ULA

Os testes realizados para avaliar a eficiência do multiplicador foram:

- $0000 \times 0000 = 0000$ ($0 \times 0 = 0$, flag ZERO)
- $0011 \times 1100 = 100100$ ($3 \times 12 = 36$, flag OVERFLOW)
- $0010 \times 0110 = 1100$ ($2 \times 6 = 12$)
- $1010 \times 1010 = 1100100$ ($10 \times 10 = 100$, flag OVERFLOW)
- $1100 \times 1000 = 1100000$ ($12 \times 8 = 96$, flag OVERFLOW)
- $1101 \times 1101 = 10101001$ ($13 \times 13 = 169$, flag OVERFLOW)

Utilizando a ferramenta *Simulation* do *Quartus*, é possível confirmar os resultados dos testes (21)

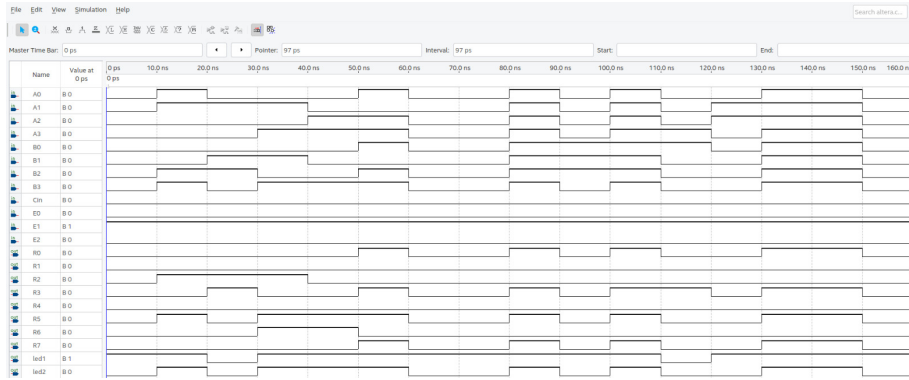


Figura 21 – Teste multiplicador de 4 bits. Fonte: Autoria Própria. 2025.

4.1.4 MÓDULO DIVISOR DE 4 BITS NA ULA

Os testes realizados para avaliar a eficiência do divisor foram:

- $1100 \div 0011 = 0100$ ($12 \div 3 = 4$)
- $1110 \div 0110 = 0010$ ($14 \div 6 = 2$)
- $1100 \div 0000 = 1111$ ($12 \div 0 = 15$, flag ERRO)
- $0000 \div 0000 = 1111$ ($0 \div 0 = 15$, flag ERRO)
- $0000 \div 1010 = 0000$ ($0 \div 10 = 0$, flag ZERO)

Utilizando a ferramenta *Simulation* do *Quartus*, é possível confirmar os resultados dos testes (22).

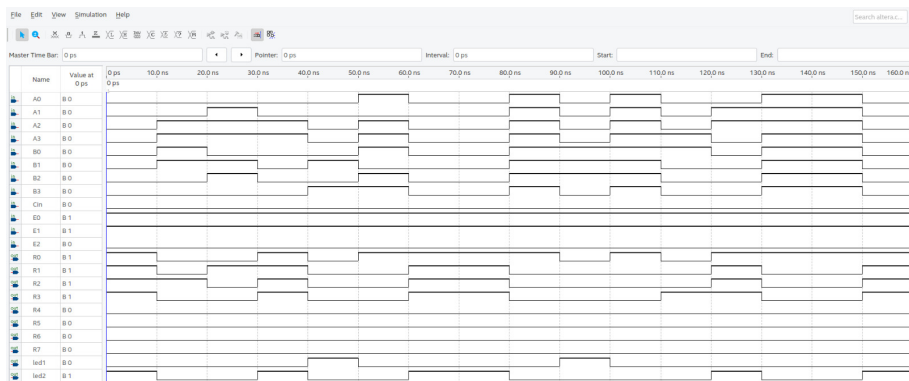


Figura 22 – Teste divisor de 4 bits. Fonte: Autoria própria. 2025.

4.1.5 MÓDULO OPERAÇÃO AND DE 4 BITS NA ULA

Os testes realizados para avaliar a operação AND foram:

- $0001 \text{ AND } 0001 = 0001$ (1)
- $1111 \text{ AND } 1001 = 1001$ (9)

- $0100 \text{ AND } 0100 = 0100$ (4)

Utilizando a ferramenta *Simulation* do *Quartus*, é possível confirmar os resultados dos testes (23).

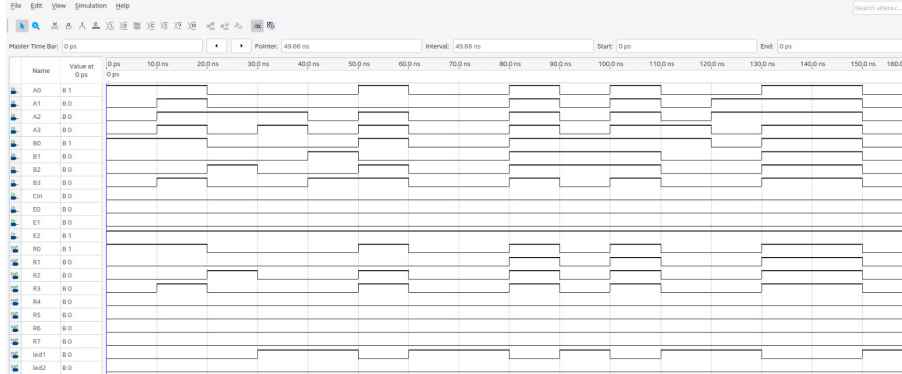


Figura 23 – Teste operação AND de 4 bits. Fonte: Autoria própria. 2025.

4.1.6 MÓDULO OPERAÇÃO OR DE 4 BITS NA ULA

Os testes realizados para avaliar a operação OR foram:

- $0001 \text{ OR } 0001 = 0001$ (1)
- $1101 \text{ OR } 1001 = 1101$ (13)
- $0100 \text{ OR } 0101 = 0101$ (5)

Utilizando a ferramenta *Simulation* do *Quartus*, é possível confirmar os resultados dos testes (24).

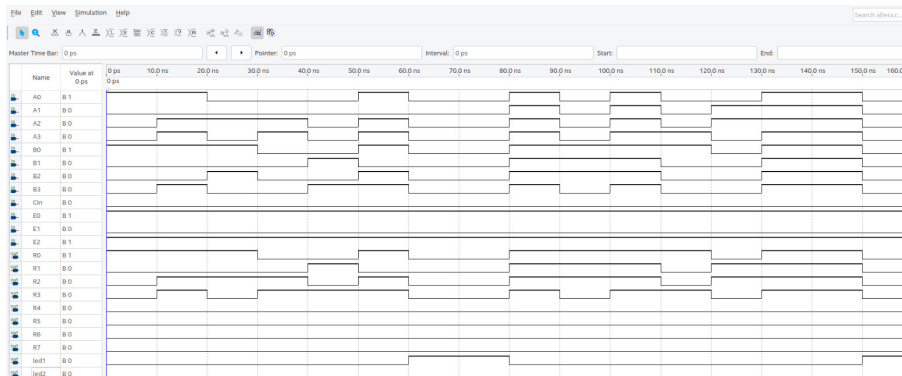


Figura 24 – Teste operação OR de 4 bits. Fonte: Autoria própria. 2025.

4.1.7 MÓDULO OPERAÇÃO XOR DE 4 BITS NA ULA

Os testes realizados para avaliar a operação XOR foram:

- $0101 \text{ XOR } 1010 = 1111$ (15)

- $1001 \text{ XOR } 1001 = 0000$ (0, flag ZERO)
- $0100 \text{ XOR } 0101 = 0001$ (1)

Utilizando a ferramenta *Simulation* do *Quartus*, é possível confirmar os resultados dos testes (25).

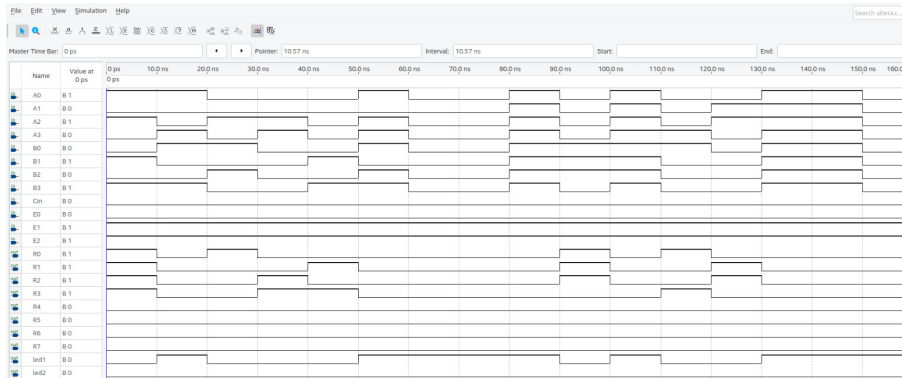


Figura 25 – Teste operação XOR de 4 bits. Fonte: Autoria própria. 2025.

5 CONCLUSÃO

O protótipo da ULA foi implementado na placa FPGA DE10-Lite utilizando Verilog estrutural no software Quartus, com os módulos organizados de acordo com cada operação e um multiplexador controlando a seleção do resultado final. Flags de condição foram incluídas e testadas para diferentes cenários, garantindo que sinais como zero, carry e overflow fossem acionados corretamente.

Os resultados das operações puderam ser visualizados diretamente nos LEDs da placa, permitindo a verificação prática do funcionamento da ULA. O desenvolvimento demonstra a eficiência da modularização em projetos digitais e ilustra conceitos fundamentais de arquitetura de computadores, conforme referência consolidada na literatura [MANO, 2012](#).

6 REFERÊNCIAS BIBLIOGRÁFICAS

Referências

IDOETA, F. G.; CAPUANO, L. *Elementos da Eletrônica Digital*. 41. ed. São Paulo: Saraiva, 2012. Disponível em: <https://pt.scribd.com/document/557897594/Elementos-de-Eletronica-Digital-CAPUANO-Francisco-Gabriel-IDOETA-Ivan-Valeije>. Acesso em: 29 set. 2025. Nenhuma citação no texto.

MANO, M. M. *Digital Design*. 5. ed. São Paulo: Pearson, 2012. Disponível em: <https://www.auhd.edu.ye/upfiles/elibrary/Azal2020-01-22-12-16-48-75016.pdf>. Acesso em: 29 set. 2025. Nenhuma citação no texto.

CHARLIE-COLEMAN, A. *Kmap Solver*. Disponível em: <<https://www.charlie-coleman.com/experiments/kmap/>>. Acesso em: 29 set. 2025. Nenhuma citação no texto.

BLOGSPOT.COM. Representação da multiplicação binária. Disponível em: <<https://www.blogspot.com>>. Acesso em: 29 set. 2025. Nenhuma citação no texto.

ACTUALIDAD TECNOLOGICA. Display de 7 segmentos. Disponível em: <<https://www.actualidadtecnologica.com>>. Acesso em: 29 set. 2025. Nenhuma citação no texto.